

Graph Embeddings su PharMeBiNet Analisi Comparativa:

GraphSAGE · TransE · HashGNN



Sandi Russo



Università degli Studi di Messina



Scienze Informatiche



Anno Accademico 2025/2026



Relatore: Prof. Antonio Celesti

Agenda

Contesto e Dataset

GraphSAGE

TransE

HashGNN

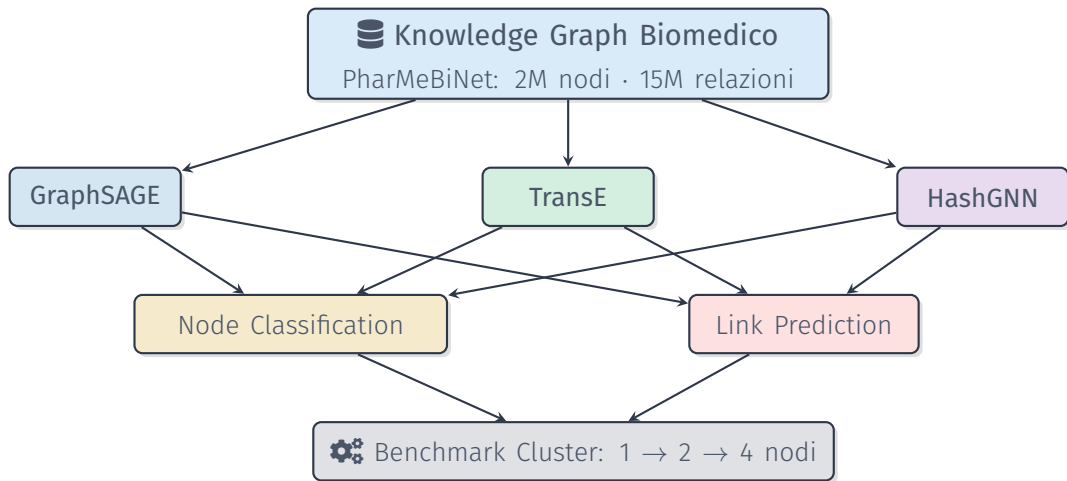
Comparazione Teorica

Node Classification vs Link Prediction

Predizione su PharMeBiNet

Benchmark Prestazionale (1-2-4 Nodi)

Contesto e Dataset



💡 Finalità dell'Analisi

L'obiettivo di questo lavoro è valutare in modo strutturato quale algoritmo di embedding risulti più idoneo all'estrazione di conoscenza da reti eterogenee, analizzandone sia le prestazioni qualitative sia quelle computazionali al variare del numero di nodi del cluster.

⚠ Domanda di Ricerca

Quale algoritmo offre il miglior equilibrio tra **qualità**, **accuratezza** e **scalabilità** su un KG biomedico fortemente eterogeneo?

⚙ Stack Tecnologico

- 🗄 Neo4j + APOC + GDS
- 🐍 Python: PyG, graphdatascience
- ☁ Azure Lab Services (8c, 32GB/VM)
- 🚢 Docker Compose + Ansible

Pharmaceutical Mechanistic Biomedical Network

Knowledge Graph biomedico open-source che integra **oltre 20 database** (DrugBank, ChEMBL, UniProt, OMIM, ...).

~2M

Nodi

66 tipi

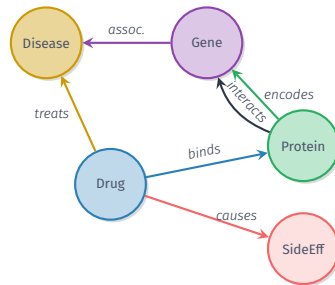
~15M

Relazioni

208 tipi

Sfida Principale

L'eterogeneità di nodi e relazioni è il fattore discriminante nella scelta dell'algoritmo.



GraphSAGE

Definizione

Graph SAmple and aggreGatE

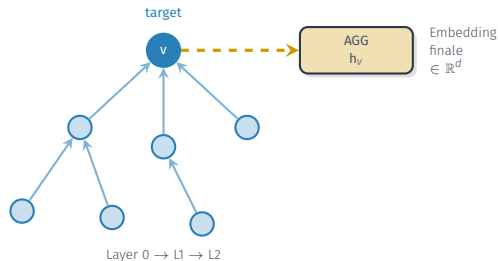
Hamilton et al., 2018

Algoritmo **induttivo**: anziché memorizzare un embedding per ogni nodo, apprende una **funzione di aggregazione** che generalizza anche a nodi *non visti* durante il training.

Idea Chiave

Il vicinato di un nodo **descrive** il nodo stesso.

Dimmi chi sono i tuoi vicini e ti dirò chi sei.



GraphSAGE – Come Funziona



⚙️ Aggregatori disponibili in GDS

- ✓ **Mean:** media vettori vicini — veloce, leggero
- ✓ **Pool:** max-pooling — più espressivo, lento

⌘ In Neo4j GDS

```
gds.beta.graphSage.train()  
gds.beta.graphSage.stream()  
Param: featureProperties, sampleSizes,  
embeddingDimension, aggregator
```

GraphSAGE – PRO e CONTRO

PRO

- ✓ **Induttivo**: generalizza su nodi non visti
- ✓ Usa le **feature dei nodi**
- ✓ Ottimo per **Node Classification**
- ✓ Buona **scalabilità** in cluster GDS
- ✓ Supporta grafi **multi-label**

CONTRO

- ✗ **Ignora tipo di relazione**
- ✗ Non per KG **eterogenei** nativi
- ✗ LP richiede **step aggiuntivo**
- ✗ Nodi isolati → zero degenerati

Caso d'uso ideale

Grafi **omogenei** o **debolmente eterogenei** con feature numeriche ricche, es. reti sociali (Reddit, PPI), grafi di citazioni (Cora, Citeseer), sistemi di recommendation.

TransE

Definizione

Translating Embeddings

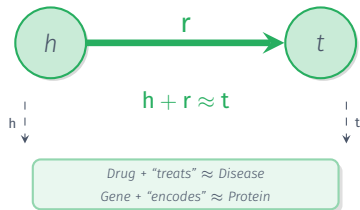
Bordes et al., 2013

Modello **transductive** per Knowledge Graph Embedding. Rappresenta entità e relazioni nello stesso spazio applicando una **traduzione geometrica**:

$$h + r \approx t$$

Idea Chiave

Ogni **tipo di relazione** è un vettore di traslazione nello spazio $\mathbb{R}^d$. Da “Drug” nella direzione “treats”, arrivo a “Disease”.



TransE – Come Funziona



Training con PyG

```
from torch_geometric.nn import
TransE
model = TransE(num_nodes,
               num_relations,
               hidden_channels=50)
loss = model.loss(head, rel, tail)
```

Prediction con GDS

```
gds.model.transe.create(G, "emb",
...)
model.predict_stream(
    source_node_filter=...,
    top_k=3)
```

PRO

- ✓ Apprende embedding per **tipo relazione**
- ✓ Progettato per **Link Prediction** su KG
- ✓ Ideale per grafi **eterogenei** (208 tipi)
- ✓ Ottimi risultati su benchmark KG

CONTRO

- ✗ **Transductive**: non generalizza
- ✗ Difficoltà con relazioni **N:N**
- ✗ **Non usa feature** dei nodi
- ✗ Training **separato** da Neo4j (PyG)

Caso d'uso ideale

Knowledge Graph con relazioni eterogenee e task di **completamento KG**: drug-target interaction, gene-disease association. Dataset: FB15k-237, WN18RR, UMLS, PharMeBiNet.

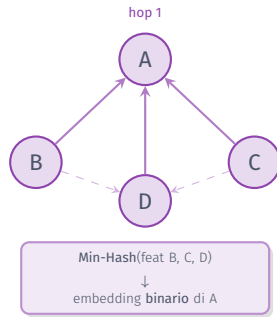
HashGNN

Definizione

Hashing-based Graph Neural Network
Algoritmo **unsupervised**: nonostante il nome, **non è un modello neurale**. Usa **Locality Sensitive Min-Hashing (LSH)** invece del gradiente. Ispirato al message passing dei GNN.

Idea Chiave

Le hash function sono **casuali e indipendenti** dai dati $\Rightarrow$ **nessun training necessario**.
Velocissimo e scalabile anche senza GPU.



HashGNN – Come Funziona



</> In Neo4j GDS

```
gds.hashgnn.mutate(G, {
  iterations: 4,
  heterogeneous: true,
  embeddingDensity: 512
})
```

🧩 Eterogeneità Nativa

Con `heterogeneous: true` HashGNN gestisce **automaticamente** nodi e relazioni di tipo diverso, senza proiezioni aggiuntive.

HashGNN – PRO e CONTRO

PRO

- ✓ **Nessun training:** hash random → istantaneo
- ✓ Supporta grafi **eterogenei** nativamente
- ✓ Ottimo per **Node Classification** multimodale
- ✓ **Scalabilità eccellente** in cluster GDS
- ✓ Funziona senza GPU

Caso d'uso ideale

Grafi eterogenei con feature binarie disponibili: IMDB, grafi biomedici con **fingerprint molecolari**, ontologie. Ideale come **baseline rapida** prima di modelli più costosi.

CONTRO

- ✗ Richiede feature **binarie** o pre-processing
- ✗ **Non semantico:** ignora il *tipo* delle rel.
- ✗ Embedding **binari**: meno espressivi
- ✗ Non adatto per **Link Prediction** semantica

Comparazione Teorica

Comparazione Visiva: i Tre Algoritmi

GraphSAGE

- ✓ Induttivo
- ✓ Feature nodi
- ✓ Node Classification
- ~ Link Prediction
- ✗ Relazioni semantiche
- ✗ KG eterogenei

Velocità: Media

GPU: Utile

Scaling: Buono

Best: NC con feature

TransE

- ✗ Transductive
- ✗ Feature nodi
- ~ Node Classification
- ✓ Link Prediction
- ✓ Relazioni semantiche
- ✓ KG eterogenei

Velocità: Lenta

GPU: Sì (PyG)

Scaling: Medio

Best: Link Prediction

HashGNN

- ✓ Induttivo
- ✓ Feature nodi (bin.)
- ✓ Node Classification
- ✗ Link Prediction
- ~ Relazioni semantiche
- ✓ KG eterogenei

Velocità: Molto veloce

GPU: No

Scaling:

Best: Velocità

✓ vantaggio ~ neutro ✗ svantaggio

Node Classification vs Link Prediction

Definizione

Dato un nodo v , predire la sua **label/categoria** (es. tipo di farmaco, funzione proteica, categoria di malattia).

GraphSAGE

- ✓ Embedding multi-hop
- ✓ Inductive (nodi nuovi)
- ✓ Feature continue
- ~ Richiede training

Metrica: F1-Macro

TransE

- ~ Transductive
- ✗ No feature nodi
- ~ Classificatore extra
- ~ Utile se rel. discrim.

Metrica: Accuracy

HashGNN

- ✓ Nativo per KG etero
- ✓ Istantaneo, no GPU
- ✓ Pipeline GDS
- ✗ Solo feature binarie

Metrica: F1-Macro

Definizione

Predire se esiste (o esisterà) una **relazione specifica** tra due nodi (es. drug-target binding, gene-disease, drug-drug interaction).

GraphSAGE

- ~ Op. ad hoc su emb.
- ✓ Vicinato strutturale
- ✗ No tipo relazione
- ~ Solo rel. omogenee

Metrica: AUC-ROC

TransE

- ✓ Progettato per LP KG
- ✓ Score $\|h+r-t\|$
- ✓ 208 rel. semantiche
- ✗ Difficoltà N:N

Metrica: Hits@10, MRR

HashGNN

- ✗ Non pensato per LP
- ✗ No semantica relazioni
- ~ Solo feature extractor
- ✗ Emb. binari limitanti

Metrica: AUC-ROC

Raccomandazione per Node Classification

HashGNN come eccellente baseline rapida. **GraphSAGE** per raggiungere le performance massimali, specialmente se sono disponibili feature numeriche continue. **TransE** da usare solo indirettamente come estrattore di feature strutturali.

Raccomandazione per Link Prediction

TransE è il candidato naturale e indiscusso per la Link Prediction su un Knowledge Graph eterogeneo come PharMeBiNet. È l'unico a modellare esplicitamente la semantica e la direzione di tutti i 208 tipi di relazioni.

Predizione su PharMeBiNet

⚠ Nota Metodologica

Predizioni basate su analisi **teorica**. I risultati sperimentali verificheranno o smentiranno queste ipotesi: è la parte centrale della tesi.

Node Classification

1. GraphSAGE ↑

Feature continue + multi-hop → cattura contesto biologico

2. HashGNN →

Fingerprint/ontologie binarie. Ottima baseline rapida

3. TransE ↓

Non progettato per classificazione diretta

Link Prediction

1. TransE ↑

208 tipi relazione: semantica vettoriale nativa.
Ideale per drug-target, gene-disease

2. GraphSAGE →

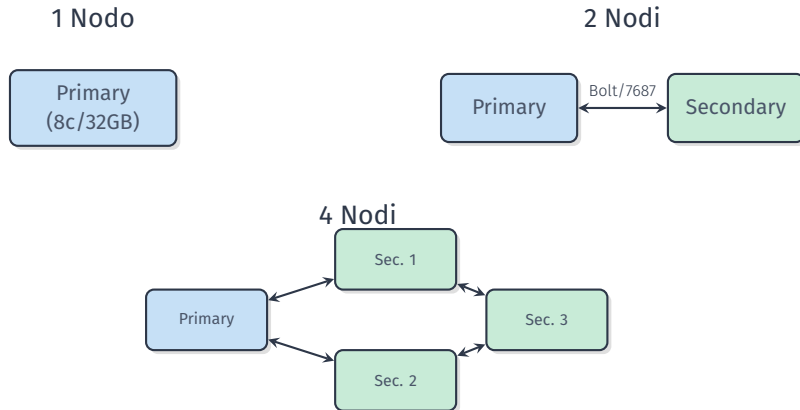
Perde informazione sulla direzione semantica

3. HashGNN ↓

Non adatto a LP semantica su KG

Benchmark Prestazionale (1-2-4 Nodi)

Setup del Cluster: Architettura



Setup del Cluster: Stack e Workflow

Azure Lab Services

-  Template: Win 10 Ent. LTSC 2021
-  8 core · 32 GB RAM per VM
-  Docker Compose per Neo4j
-  Ansible: deploy automatico su tutte le VM

Workflow di Deploy

1. Template VM → Publish
2. Ansible configura Primary + Secondary
3. Apre porte firewall + WSL portproxy
4. Avvia **docker compose up** su ogni nodo
5. Verifica cluster da Primary

Ipotesi Prestazionali sul Cluster

GraphSAGE

- ~ GDS partial distrib.
- ✓ Scaling **lineare** NC
- ~ RAM: alta (feat float)
- ✓ Parallelismo nativo
- ~ Bottleneck: multi-hop

Speedup: buono (1→4)

TransE

- ✓ Training distr. PyG
- ~ Scaling parziale
- ~ RAM: alta (KGE)
- ~ Query post-training
- ✗ Bottleneck: PyG

Speedup: medio (1→4)

HashGNN

- ✓ Istantaneo, no train
- ✓ Scaling **ottimo**
- ✓ RAM: bassa (bit vett.)
- ✓ Parallelismo GDS
- ~ Bottleneck: hashing

Speedup: eccellente

Predizione Globale

HashGNN sarà il più veloce in assoluto. **TransE** beneficia meno della distribuzione (bottleneck PyG). **GraphSAGE** mostrerà lo scaling più lineare su Node Classification.

Metriche di Qualità ML

Node Classification:

- ★ Accuracy, F1-Macro, F1-Weighted
- ★ Confusion Matrix per tipo nodo

Link Prediction:

- ★ Mean Rank (MR), MRR
- ★ Hits@1, Hits@3, Hits@10
- ★ AUC-ROC

Metriche Prestazionali

- ★ Training time (ms) su 1, 2, 4 nodi
- ★ Throughput (embedding/sec)
- ★ Memory footprint GDS in-memory
- ★ Speedup: $S_n = T_1/T_n$
- ★ Efficienza: $E_n = S_n/n$

Workflow Sperimentale

1. EDA su PharMeBiNet
2. Generazione embedding (3 algoritmi)
3. Pipeline ML: NC + LP
4. Benchmark 1/2/4 nodi → Analisi

Node Classification

1. GraphSAGE $\geq$
HashGNN > TransE

Link Prediction

1. TransE $\gg$ Graph-SAGE > HashGNN

Velocità / Scalabilità

1. HashGNN > Graph-SAGE > TransE

Best-All-Round Previsto

Non esiste un vincitore assoluto: il migliore dipende dalla **task**. La tesi dimostrerà empiricamente i trade-off su PharMeBiNet.

Prossimi Passi

1.  Deploy Cluster su Azure
2.  Caricare PharMeBiNet + EDA
3.  Pipeline GDS/PyG (3 algo)
4.  Benchmark 1 $\rightarrow$ 2 $\rightarrow$ 4 nodi
5.  Analisi risultati + stesura